Indian Institute of technology - Jodhpur

MLOPS Group Project Report

PROJECT TOPIC - IMDB SENTIMENT ANALYSIS

Prof. Dr. Hardik Jain

SUBMITTED BY - MLOPS GROUP 26

MEMBERS -
Karan Singh (G25AIT2048),
Anshuman Yadav(G25AIT2014),
Shubham Shukla(G25AIT2104)

Introduction.

Sentiment analysis is a Natural Language Processing (NLP) task that aims to automatically determine the emotional tone or opinion expressed in textual data. With the rapid growth of user-generated content on online platforms, sentiment analysis has become an important tool for understanding customer feedback, product reviews, and public opinion. Movie review classification is one of the most widely studied sentiment analysis problems, where reviews are categorized as either positive or negative based on their content.

Dataset

The dataset used in this project is the IMDb Movie Reviews dataset, obtained from the Hugging Face Datasets library. The dataset contains movie reviews collected from the Internet Movie Database (IMDb) and is widely used as a benchmark for sentiment analysis tasks.

The dataset consists of 50,000 English-language movie reviews, evenly distributed between positive and negative sentiment classes. It is pre-divided into 25,000 training samples and 25,000 testing samples, ensuring a balanced class distribution and eliminating class imbalance concerns during model training.

Each record in the dataset contains two attributes:

text: The movie review written by a user.
label: The sentiment label associated with the review, where:
0 = Negative Sentiment
1 = Positive Sentiment

During the exploratory data analysis phase, the dataset was examined for missing values, duplicate records, class distribution, review length statistics, and text quality. Basic text cleaning was performed to remove unnecessary whitespace and normalize the input data while preserving the semantic information required for transformer-based models.

Model

This project utilizes DistilBERT (Distilled Bidirectional Encoder Representations from Transformers), a compact transformer-based language model developed by Google Research and later optimized by Hugging Face. DistilBERT is created through knowledge distillation, a process in which a smaller student model learns to replicate the behavior of a larger BERT model.

The pre-trained model selected for this project is distilbert-base-uncased, which contains approximately 66 million parameters and supports case-insensitive English text processing. Compared to the original BERT model, DistilBERT achieves nearly equivalent performance while being significantly smaller, faster, and more computationally efficient.

Several factors motivated the selection of DistilBERT for this project:

Strong performance on text classification and sentiment analysis tasks.
Reduced training and inference time compared to larger transformer models.
Lower memory requirements, making it suitable for deployment and experimentation on Kaggle GPU environments.
Seamless integration with the Hugging Face Transformers ecosystem.
Availability of pre-trained weights that capture rich contextual language representations.

For sentiment classification, a sequence classification head was added to the pre-trained DistilBERT encoder. The model was then fine-tuned on the IMDb dataset using supervised learning, allowing it to adapt its learned language representations specifically to the task of positive and negative sentiment prediction.

The final fine-tuned model serves as the core component of the sentiment analysis pipeline and is deployed through a production-oriented MLOps workflow involving experiment tracking, model versioning, containerization, and automated deployment.

Project structure and Setup

The project was developed following a modular MLOps architecture to ensure reproducibility, maintainability, and scalability. Development was performed locally using the **uv package manager** for Python dependency management and virtual environment creation. The project source code was version-controlled using Git and hosted on GitHub to facilitate collaboration, code tracking, and automated workflows.

The overall workflow consisted of local development, cloud-based experimentation, model tracking, and model deployment. Data preprocessing, model training scripts, evaluation modules, and API services were organized into separate directories to maintain a clear separation of concerns.

The project structure is shown below:

```
ML-Group-Project-GRP-26/
│
├── src/
│   ├── data–.py
│   ├── preprocess.py
│   ├── train.py
│   └── evaluate.py
│
├── api/
│   └── app.py
│
├── configs/
│   ├── v1.yaml
│   ├── v2.yaml
│   └── v3.yaml
│
├── notebooks/
│   └── imdb_eda.ipynb
│
├── Model/
│   └── Trainer/
│
├── Reports/
│
├── requirements.txt
├── Dockerfile
├── README.md
└── .gitignore
```

The IMDb dataset was obtained from the Hugging Face Datasets library and processed through a dedicated preprocessing pipeline. Exploratory Data Analysis (EDA) was conducted using Jupyter notebooks to understand dataset characteristics and prepare cleaned datasets for training.

Model experimentation and fine-tuning were executed on Kaggle GPU environments, enabling efficient training of transformer-based models. Multiple experiment configurations were evaluated by varying hyperparameters such as learning rate, batch size, and number of training epochs.

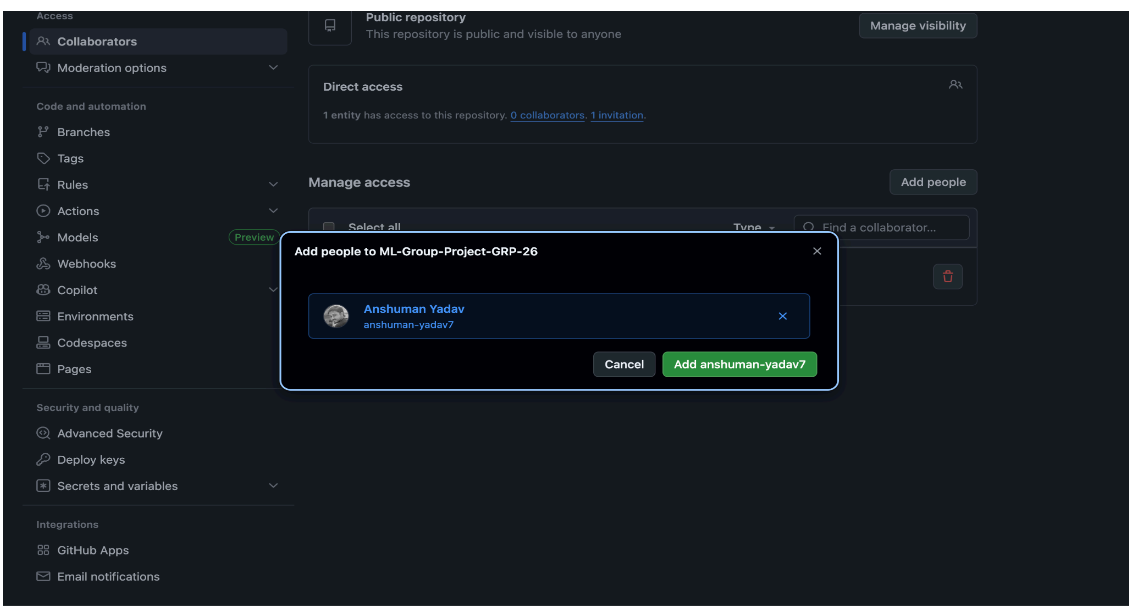
Weights & Biases (W&B) was integrated throughout the training process to track experiments, monitor training metrics, compare model versions, and store run artifacts. This facilitated systematic evaluation and selection of the best-performing model configuration.

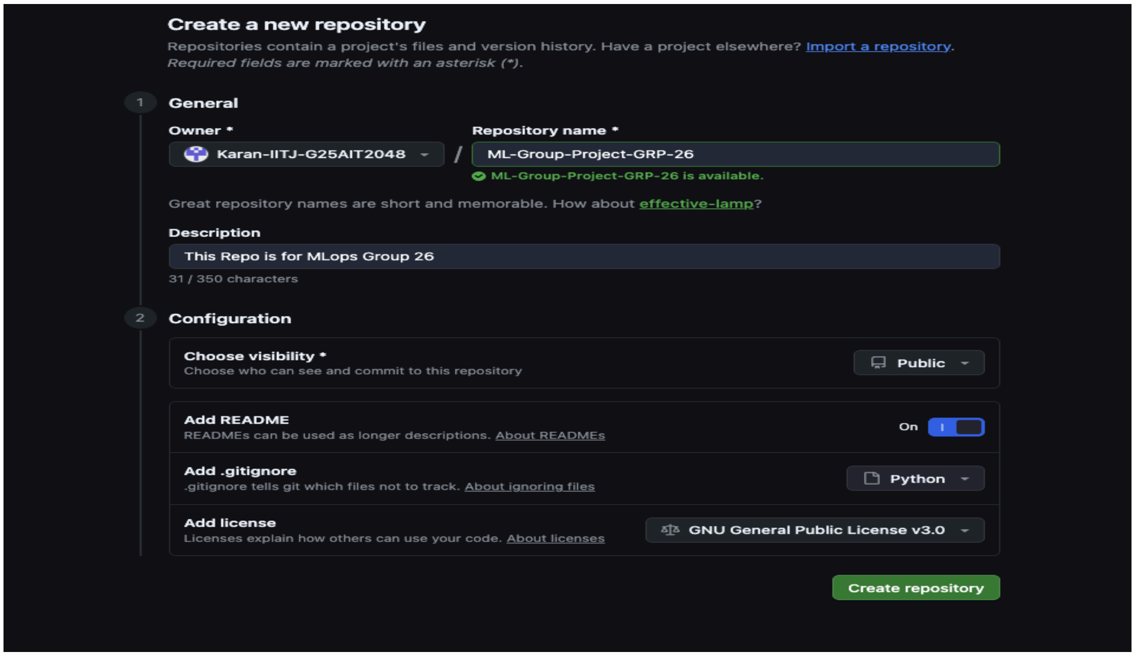
The final trained model was uploaded to the Hugging Face Hub, providing centralized model versioning and easy access for inference and deployment. The model repository serves as the primary source for downloading and loading the production-ready sentiment classification model.

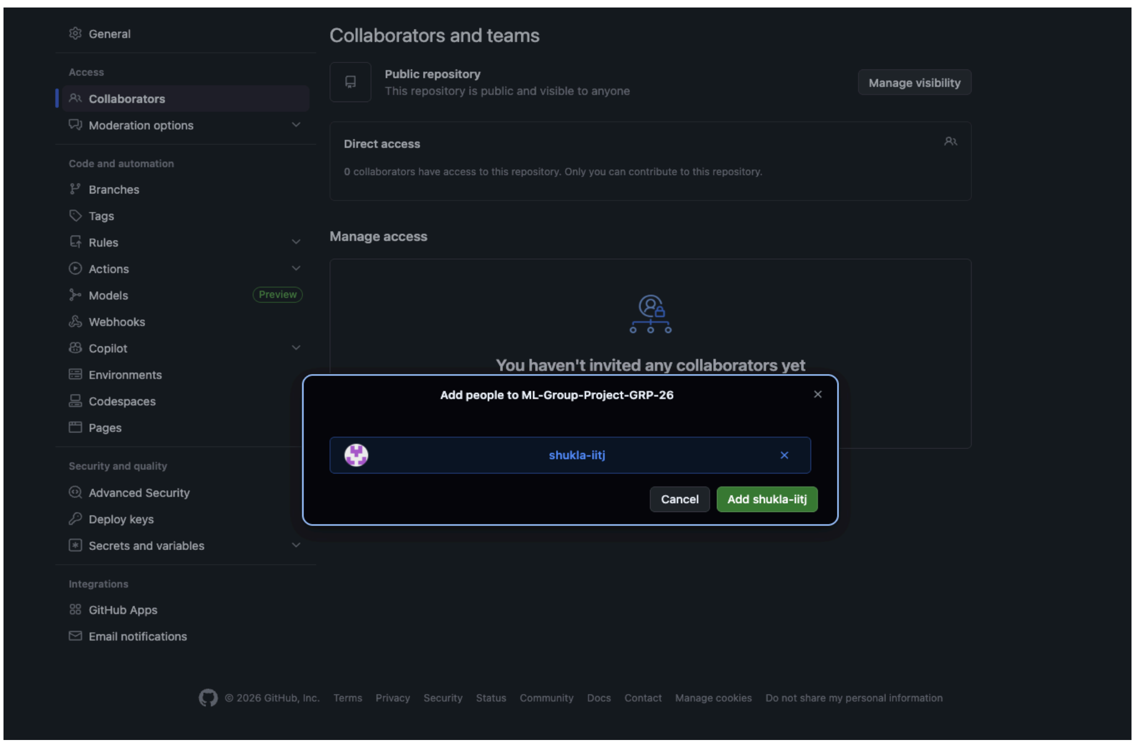
For deployment, a FastAPI inference service was developed and containerized using Docker, enabling consistent execution across environments. Continuous Integration and Continuous Deployment (CI/CD) workflows were implemented through GitHub Actions to automate testing, training, and deployment-related tasks.

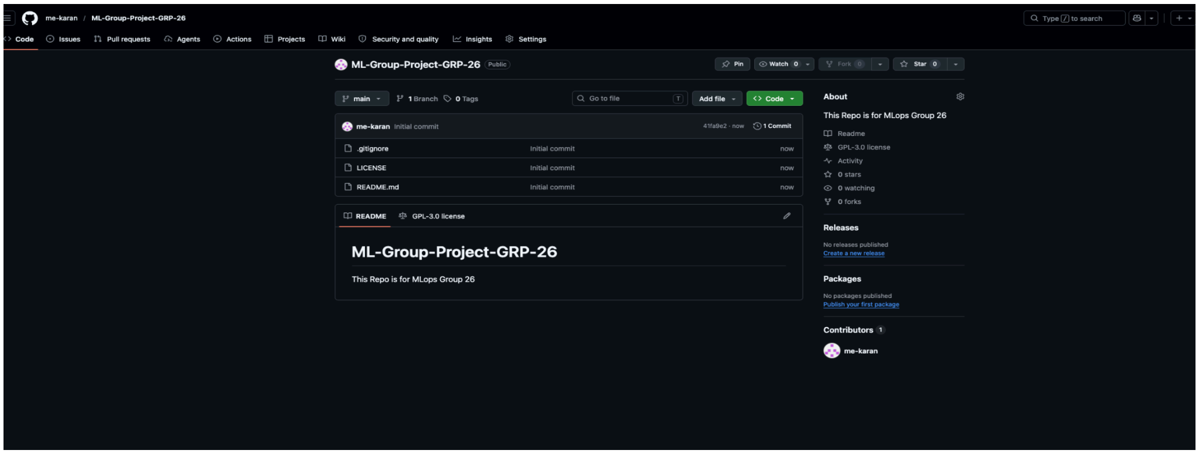
GitHub and Github Actions

Github repository - https://github.com/Karan-IITJ-G25AIT2048/ML-Group-Project-GRP-26



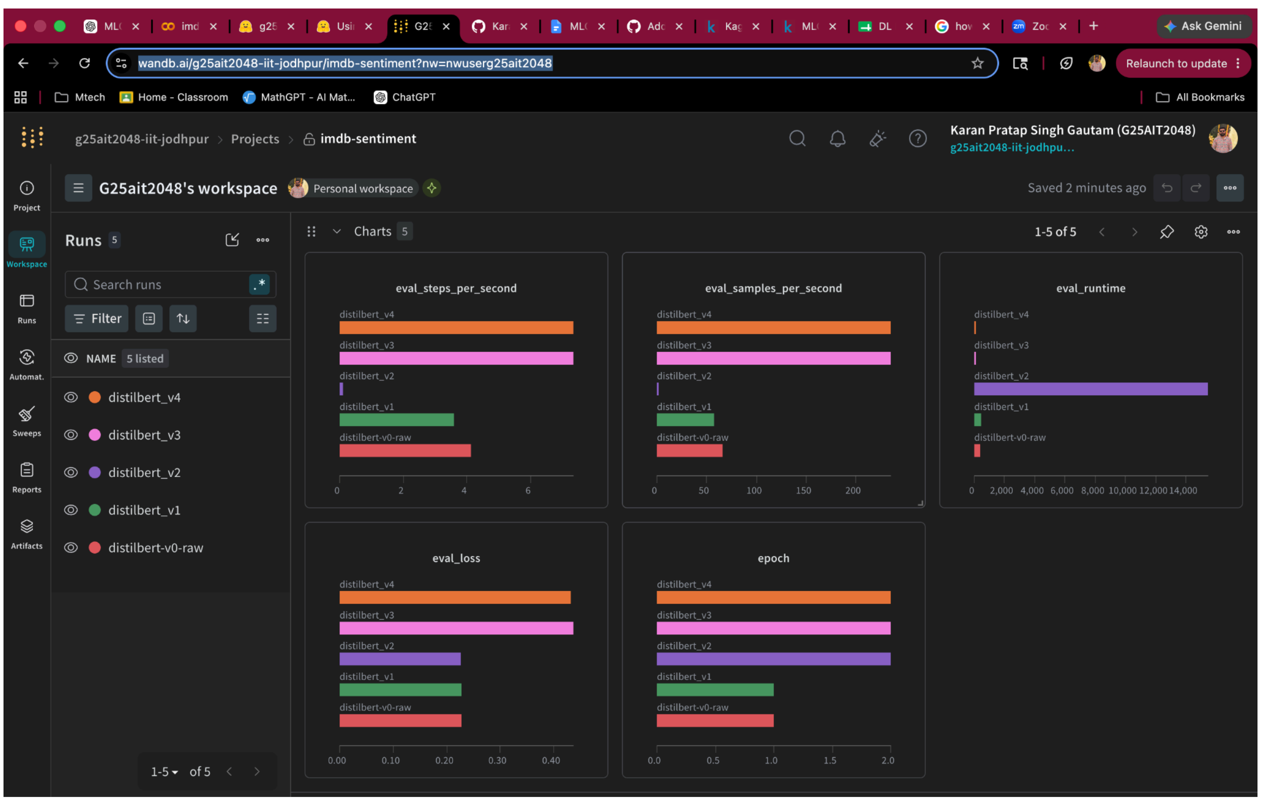

Create a new repository
Repositories contain a project's files and version history. Have a project elsewhere? Import a repository.
Required fields are marked with an asterisk (*).
1
General
Owner *
Karan-IITJ-G25AIT2048
/
Repository name *
ML-Group-Project-GRP-26
ML-Group-Project-GRP-26 is available.
Great repository names are short and memorable. How about effective-lamp?
Description
This Repo is for MLops Group 26
31 / 350 characters
2
Configuration
Choose visibility *
Choose who can see and commit to this repository
Public
Add README
READMEs can be used as longer descriptions. About READMEs
On
Add .gitignore
.gitignore tells git which files not to track. About ignoring files
Python
Add license
Licenses explain how others can use your code. About licenses
GNU General Public License v3.0
Create repository

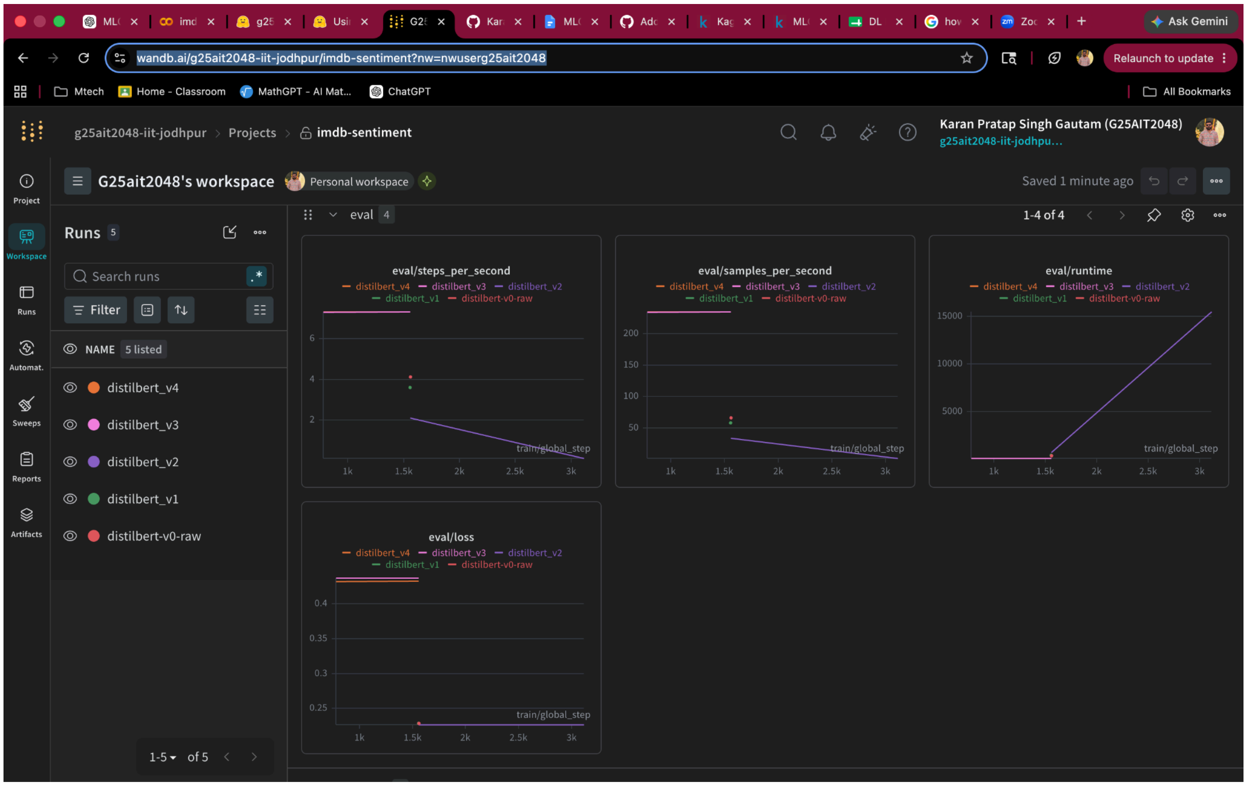

General
Access
Collaborators
Moderation options
Code and automation
Branches
Tags
Rules
Actions
Models
Preview
Webhooks
Copilot
Environments
Codespaces
Pages
Security and quality
Advanced Security
Deploy keys
Secrets and variables
Integrations
GitHub Apps
Email notifications
Collaborators and teams
Public repository
This repository is public and visible to anyone
Manage visibility
Direct access
0 collaborators have access to this repository. Only you can contribute to this repository.
Manage access
You haven't invited any collaborators yet
Add people to ML-Group-Project-GRP-26
shukla-iitj
Cancel
Add shukla-iitj
© 2026 GitHub, Inc.
Terms
Privacy
Security
Status
Community
Docs
Contact
Manage cookies
Do not share my personal information



W&B Dashboard

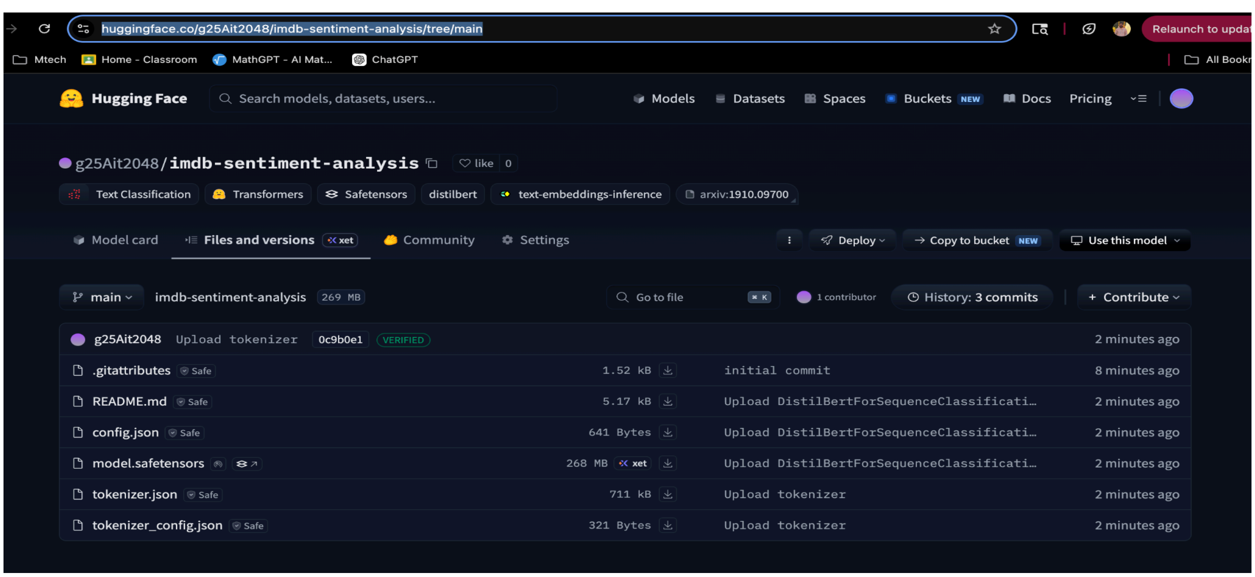
Overiview - https://wandb.ai/g25ait2048-iit-jodhpur/imdb-sentiment/overview
Dashboard - https://wandb.ai/g25ait2048-iit-jodhpur/imdb-sentiment?nw=nwuserg25ait2048





Hugging face Repo

HF Repository - https://huggingface.co/g25Ait2048/imdb-sentiment-analysis/tree/main



Results

Clean Data	Unclean Data

{
"accuracy": 0.9126,
"precision": 0.9178481730535526,
"recall": 0.90632,
"f1": 0.9120476593004065
}
	{
	"accuracy": 0.915,
	"precision": 0.9118960293414129,
	"recall": 0.9193729903536978,
	"f1": 0.915619245857017
	}

Ran 4 iterations for experimentation

Configs:

v1	run_name: distilbert_v1 learning_rate: 2e-5 batch_size: 16 epochs: 1 max_length: 256	accuracy: 0.9108 precision: 0.9073 recall: 0.9157 f1: 0.9115
v2	run_name: distilbert_v2 learning_rate: 2e-5 batch_size: 16 epochs: 2 max_length: 256	accuracy: 0.9124 precision: 0.9156 recall: 0.9092 f1: 0.9124
v3	run_name: distilbert_v3 learning_rate: 3e-5 batch_size: 16 epochs: 2 max_length: 256	accuracy : 0.9143 precision : 0.9068 recall : 0.9240 f1 : 0.9154
v4	run_name: distilbert_v4 learning_rate: 5e-5 batch_size: 16 epochs: 2 max_length: 256	accuracy : 0.9142 precision : 0.9025 recall : 0.9293 f1 : 0.9157

Best Model - V4

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to e
nable higher rate limits and faster downloads.
Loading test dataset...
Map: 100%|██████████| 24902/24902 [00:10<00:00, 2285.82 examples/s]
Map: 100%|██████████| 24800/24800 [00:10<00:00, 2328.78 examples/s]
Loading model...
Loading weights: 100%|█| 104/104 [00:00<00:00, 1589.38it/s, Materializing param=
Running predictions...
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Was as
ked to gather along dimension 0, but all input tensors were scalars; will instead unsqueeze
and return a vector.
  return super().apply(*args, **kwargs)  # type: ignore[misc]
100%|██████████| 1550/1550 [02:05<00:00, 12.38it/s]

Evaluation Metrics
------------------------------
accuracy  : 0.9142
precision : 0.9025
recall    : 0.9293
f1        : 0.9157

Reports saved to: Reports/Trainer/distilbert_v4_1

Evaluation completed successfully.
```

Best model obtained Summary - v4

{
 "_runtime": 986,
 "_step": 19,
 "_timestamp": 1781196549.2638116,
 "_wandb.runtime": 986,
 "epoch": 2,
 "eval/loss": 0.436589241027832,
 "eval/runtime": 105.6628,
 "eval/samples_per_second": 234.709,
 "eval/steps_per_second": 7.335,
 "eval_loss": 0.436589241027832,
 "eval_runtime": 105.6628,

"eval_samples_per_second": 234.709,
"eval_steps_per_second": 7.335,
"total_flos": 3298703161331712,
"train/epoch": 2,
"train/global_step": 1558,
"train/grad_norm": 2.2291500568389893,
"train/learning_rate": 0.0000011360718870346595,
"train/loss": 0.26991704940795896,
"train_loss": 0.44756224097275155,
"train_runtime": 850.357,
"train_samples_per_second": 58.568,
"train_steps_per_second": 1.832
}

Members	Contribution
Karan Singh	1. Model and Dataset selection 2. Project setup a. Local b. Kaggle 3. EDA 4. Preprocessing and cleaning 5. Evaluation and tracking 6. Github Management 7. Kaggle notebook Creation a. Repo Cloning b. Data processing c. Experiment configuration d. Experiment Evaluation 8. W&B Tracking 9. HF Repository creation & Model Uploading 10. Report Creation
Anshuman Yadav	1. Docker Setup 2. Git collaboration
Shubham Shukla	1. Github Actions CI and Interface setup 2. Git Collaboration
Suraj Samal	